

JAMES COOK UNIVERSITY

CP3407

SMART EXPENSE TRACKER WEB APPLICATION

FINAL PROJECT REPORT

Team members: Zhu Zhiwei, Jiang Jinsong and Fu Yuzhe

Campus: James Cook University Singapore

Submission date: 5 August 2026

Repository: <https://github.com/lbw138286/cp3407>

Final release: v1.0.1

Contents

- 1 Executive Summary
- 2 Problem, Users and Requirements
- 3 Agile Planning and Week 1-10 Delivery
- 4 Architecture and Design
- 5 Database and Security
- 6 Implementation and User Interface
- 7 Testing, TDD and Defect Management
- 8 Version Control and Development Tools
- 9 Acceptance Feedback and Corrective Action
- 10 Persistence and Deployment
- 11 Conclusion
- 12 References

1 Executive Summary

The Smart Expense Tracker is a full-stack web application completed across three Agile iterations. The project responds to common problems identified during user research, including forgotten small expenses, overspending, inconsistent categories, limited budget visibility and concern about the security of personal financial records. The final system provides account registration, secure login, expense creation and maintenance, category filtering, monthly budgets, budget alerts, monthly financial reports, spending trends, administrator account management and acceptance-feedback recording. The implemented architecture uses a browser-based graphical interface, a Node.js server and a relational SQLite database. This combination satisfies the project requirement for source code, a modern database and a modern graphical user interface (James Cook University, 2026b).

The final verification result is 41 automated tests passed with 0 failures and 15 system tests passed with 0 failures. GitHub Issues, feature branches, Pull Requests, GitHub Actions, teammate review and release management were used throughout the final integration. The lecturer identified one usability issue during the final demonstration: the reports were presented in a way that was too complex for normal users. The interface was then changed to use summary cards, tables and comparison bars, while raw JSON was retained only as optional technical information.

2 Problem, Users and Requirements

The project began with three representative users. The first was a university student who required simple daily entry and allowance control. The second was a working adult who required secure access and monthly spending analysis. The third was a small-business owner who required organised, editable and protected records. These

needs were converted into ten user stories and prioritised according to dependency and user value.

Table 1 Target users and project responses

User	Main need	Implemented response
University student	Simple entry and allowance control	Expense entry, monthly budgets and alerts
Working adult	Secure access and monthly analysis	Secure login, reports, categories and trends
Small-business owner	Organised and protected records	Categories, CRUD, relational storage and administrator controls

Table 2 User stories, priorities and estimates

US	Requirement	Priority	Estimate and iteration	Status
US1	Register accounts	High	Final integration	Done
US2	Secure login	High	Final integration	Done
US3	Add expenses	High	6 person-days, Iteration 1	Done

US	Requirement	Priority	Estimate and iteration	Status
US4	Edit and delete expenses	High	6 person-days, Iteration 1	Done
US5	Categorise and filter expenses	High	6 person-days, Iteration 2	Done
US6	Set monthly budgets	High	6 person-days, Iteration 2	Done
US7	Receive budget alerts	High	3 person-days, Iteration 3	Done
US8	Generate monthly reports	High	3 person-days, Iteration 3	Done
US9	Analyse spending trends	Medium	3 person-days, Iteration 3	Done
US10	Administrator account management	Medium	3 person-days, Iteration 3	Done

The implementation order was based on technical dependencies. Expense records had to exist before category analysis and budgets could be meaningful. Budgets and spending history had to exist before alerts, reports and trends could be calculated. Secure registration and login were completed as cross-cutting final-integration work because the same identity and session controls were required by all private API routes.

3 Agile Planning and Week 1-10 Delivery

The team used an iterative process with user stories, task breakdowns, estimates, project-board states, burndown evidence, velocity review, TDD, bug tracking and retrospective updates. The capacity was fixed at 12 person-days per iteration, calculated from three team members working across four available project days.

Table 3 Week 1-10 project delivery

Week	Main focus	Delivered result
1	Project selection and Agile setup	Product vision, team roles and working process
2	User research and requirements	Three user profiles and US1-US10
3	Iteration 1 planning	US3 and US4 prioritised, estimated and placed on the board
4	Iteration 1 execution	Expense CRUD and design diagrams
5	Iteration 1 review	Velocity and code-quality review
6	Iteration 2 delivery	Categories and monthly budgets
7	Automated testing	Practical test cases for US3-US7

Week	Main focus	Delivered result
8	Iteration 3 TDD	Alerts, reports, trends and mock authentication
9	Bug tracking and system test preparation	BUG-003 and regression evidence
10	Final system demonstration	US1-US10, SQLite, administration and feedback response

Table 4 Iteration plan and result

Iteration	Capacity	Scope	Result
1	12 person-days	US3 Add expenses and US4 Edit/Delete expenses	Done
2	12 person-days	US5 Categories and US6 Monthly budgets	Done
3	12 person-days	US7 Alerts, US8 Reports, US9 Trends and US10 Administration	Done

Final integration completed US1 and US2, SQLite persistence, security controls, GitHub evidence and the feedback-driven report correction.

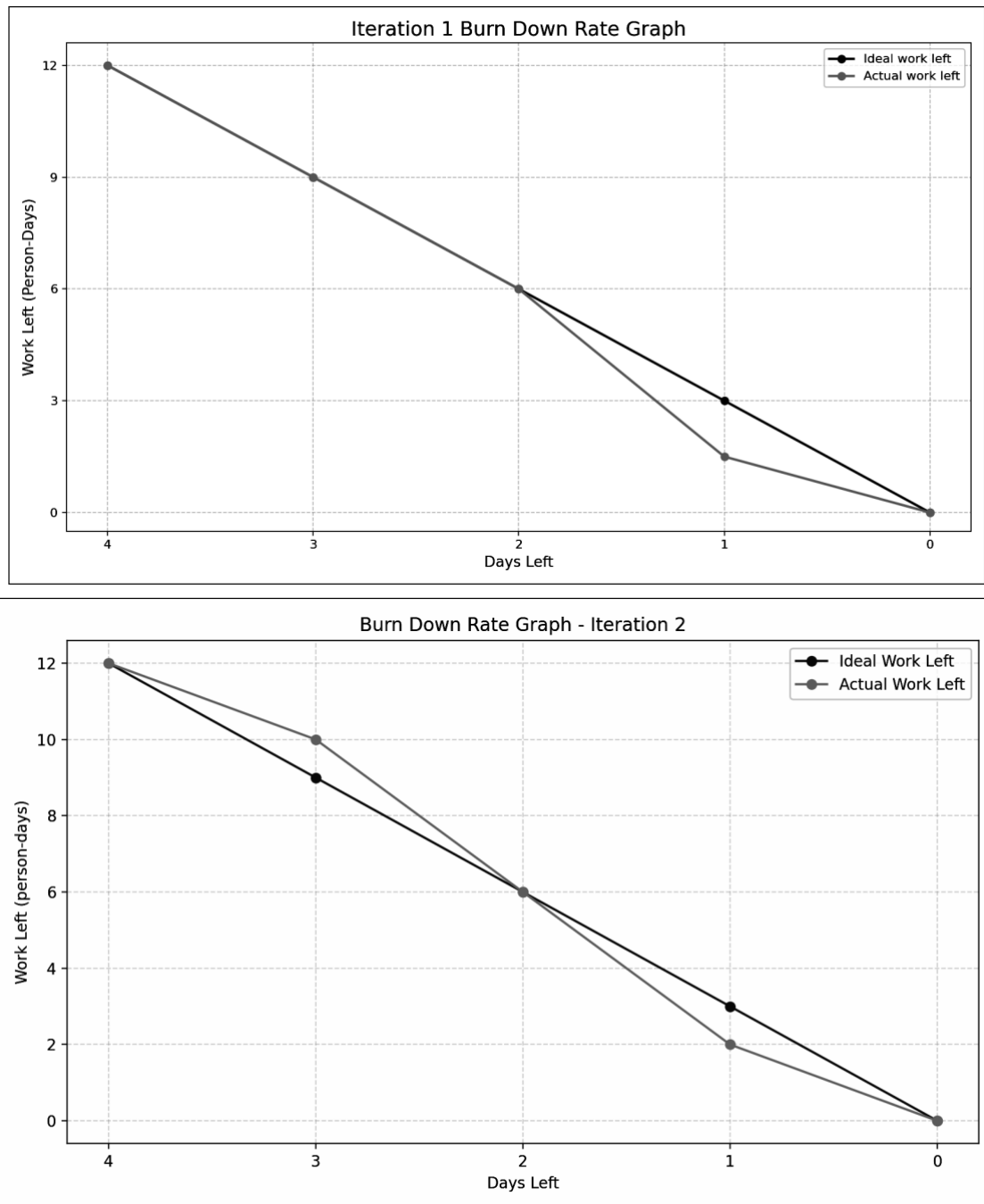


Figure 1 Iteration 1 and Iteration 2 burndown evidence

Both burndown records finished at zero remaining work. The difference between ideal and actual work was discussed during iteration reviews and was used to adjust later planning. Original weekly files are retained in the repository archive so that the development history remains available.

4 Architecture and Design

The final architecture separates the browser interface, HTTP API, security controls, business services and relational storage. This separation supports single responsibility, reduces duplicated validation and allows services and routes to be tested independently. Mermaid was used to maintain editable architecture, sequence and entity-relationship diagrams (Mermaid, n.d.).

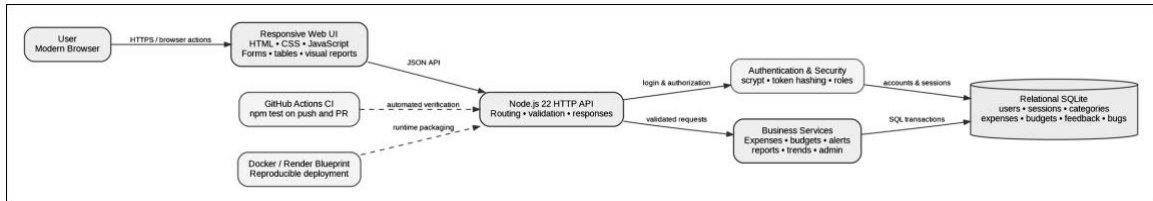


Figure 2 Layered system architecture and request flow

4.1 Relational Design

The database design uses users as the main ownership entity. A user can have many sessions, expenses and monthly budgets. Categories can be linked to many expenses. Acceptance feedback and bug records are retained as project evidence. Foreign keys are used to preserve relationships between records.

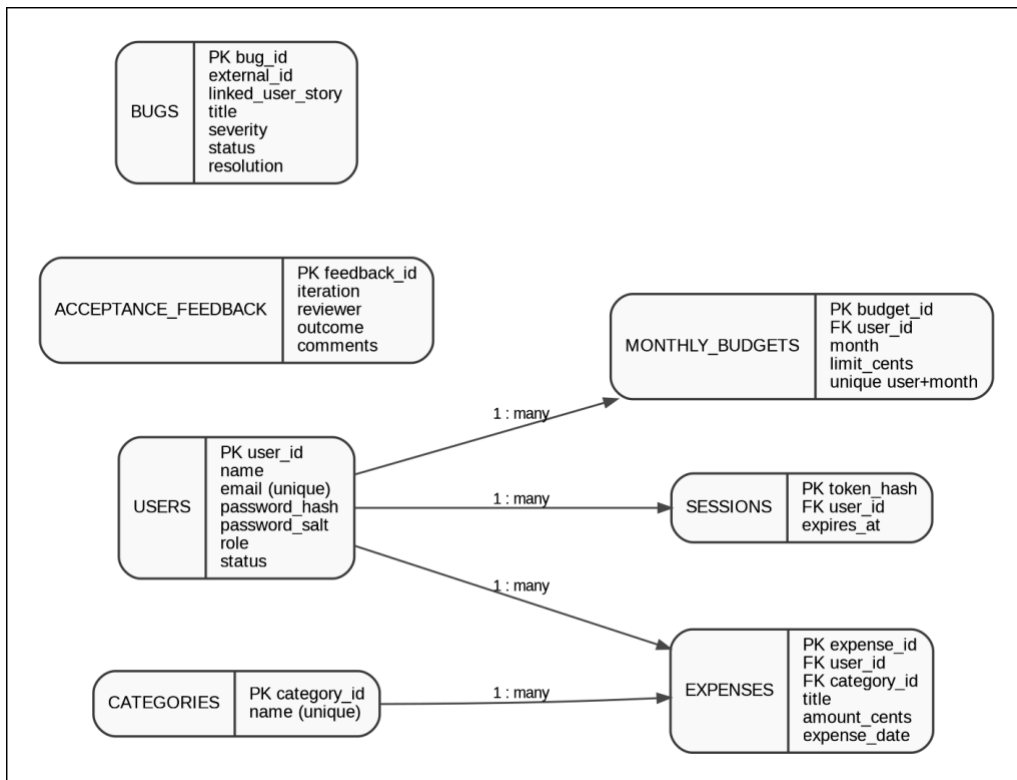


Figure 3 Relational database entity-relationship design

4.2 Behavioural Design

The sequence design shows that expense creation is not performed only in the browser. The request is sent to the server, validated, authenticated, stored in SQLite and then returned to the interface. This design supports integrity and persistence.

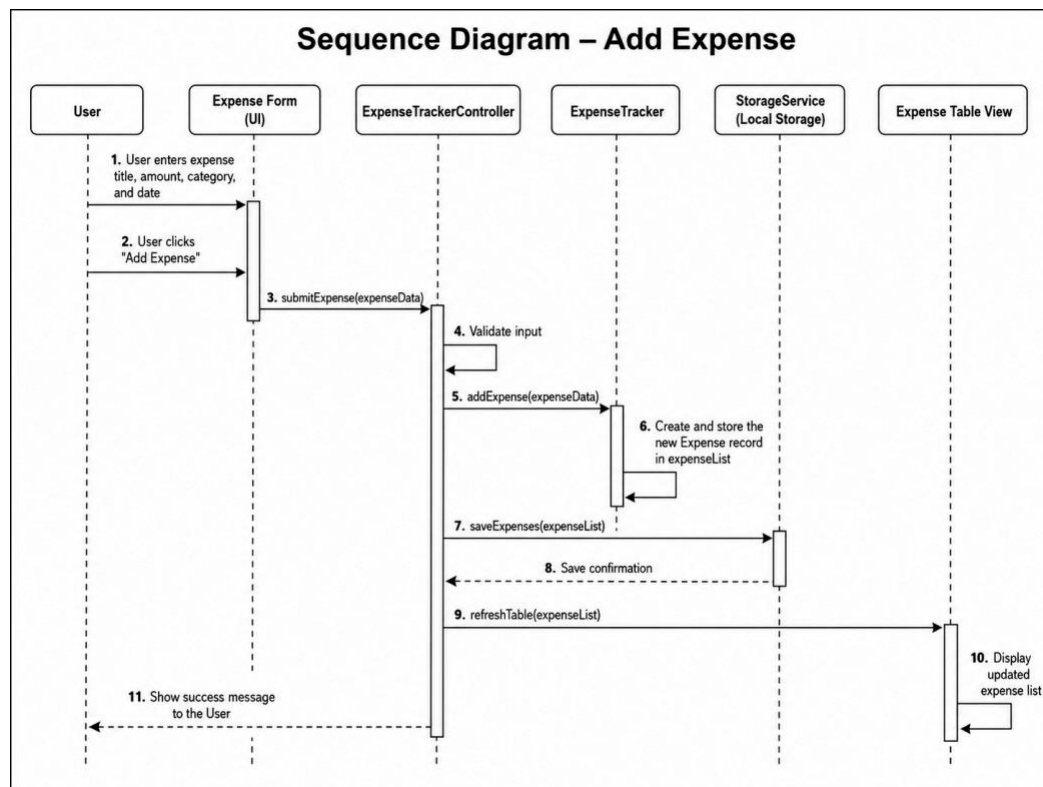


Figure 4 Add-expense sequence design

5 Database and Security

SQLite is the executable relational database used by the final application. Money is stored in integer cents to reduce floating-point storage errors. Foreign keys enforce relationships, and the database is created and seeded when the server starts. SQLite provides transactional relational storage suitable for this project scale (SQLite Consortium, n.d.).

Table 5 Database tables and controls

Table	Purpose	Main controls
-------	---------	---------------

Table	Purpose	Main controls
users	Identity and authorisation	Unique email, role, status and salted password hash
sessions	Authenticated access	Hashed token, expiry and user foreign key
categories	Controlled category names	Unique category name
expenses	User expense records	User and category foreign keys; positive amount cents
monthly_budgets	Budget by month	Unique user and month; positive limit cents
acceptance_feedback	Reviewer feedback	Iteration, reviewer, outcome and comments
bugs	Defect evidence	User story, severity, status and resolution

Security functions use Node.js cryptographic APIs and server-side session handling (Node.js, n.d.). The main controls are listed below.

Number	Statement
1	Passwords are checked for minimum strength and are stored only as random-salt scrypt hashes.
2	Plain passwords are never stored, and timing-safe comparison is used during authentication.

Number	Statement
3	Session tokens are random, while only SHA-256 token hashes are stored in SQLite.
4	Private routes require an active database session, and administrator routes require the admin role.
5	Disabled accounts cannot log in, and administrators can change account status or delete accounts.
6	HTTP responses include Content-Security-Policy, X-Content-Type-Options, X-Frame-Options and Referrer-Policy.
7	Production deployment requires HTTPS and environment-based credentials.

6 Implementation and User Interface

The browser interface communicates with authenticated server routes. Application records are not persisted in localStorage. Clear forms, validation messages, tables and action controls support both normal users and administrators. The interface is responsive and can be used in modern desktop browsers.

Table 6 Implemented application functions

Area	Implemented functions
Account	Register, login, logout, current session and disabled-account control
Expense	Add, edit, delete, validate, list and category filter

Area	Implemented functions
Budget	Set or update monthly budget and calculate safe, warning or over-budget status
Reporting	Selected-month totals, category totals, top category, summary cards and expense table
Trend	Sorted monthly totals, rounded change, direction and comparison bars
Administration	List users, update role or status, disable and delete
Feedback	Store genuine acceptance feedback in SQLite

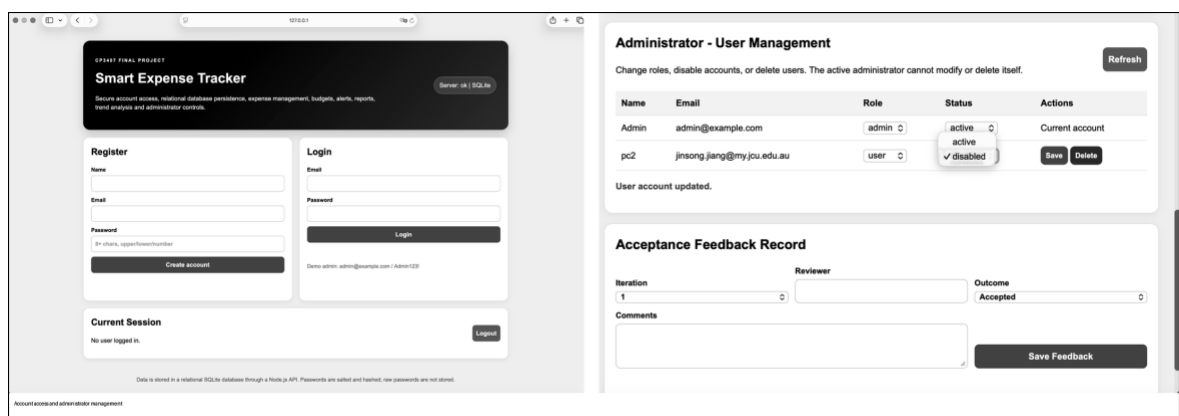


Figure 5 Account access, administrator management and feedback functions

After the final demonstration, the report interface was changed so that raw JSON is no longer the primary presentation. Monthly totals, record counts, categories and trends are now displayed in a format intended for normal users. The raw technical data remains available only in an expandable section.

7 Testing, TDD and Defect Management

The final command is `npm test`. The automated suite contains 41 tests and 0 failures.

The tests cover validation, boundary conditions, authentication, relational persistence, expense CRUD, categories, budgets, alerts, reports, trends,

administrator controls and regression defects. The final system-test result is 15 Pass, 0 Conditional Pass and 0 Fail.

Table 7 Automated test coverage

Test group	Count	Scope
Iteration 2 unit tests	15	US3-US7 expense, category, budget and alert logic
Iteration 3 unit and mock tests	12	US7-US10 reports, trends and mock authentication
Regression tests	2	BUG-003 month sorting and US9 decimal precision
Database and API integration	12	US1-US10 authentication, persistence, CRUD, reports and administration

The TDD process used during the practical work followed four repeatable stages.

Number	Statement
1	Write or update a test for the intended behaviour.
2	Run the suite and observe the failure or regression.
3	Implement the smallest valid correction.
4	Re-run unit, mock, integration and CI checks.

BUG-003 showed that unordered month totals could produce an incorrect trend comparison. The correction sorts month keys chronologically. A later US9 defect displayed a floating-point value such as 0.040000000000000001. Pull Request 27 rounded the value to two decimals and added the forty-first automated test.

[illegible]

Figure 6 Automated tests and GitHub Actions evidence

8 Version Control and Development Tools

GitHub was used for the live repository, Issues, feature branches, Pull Requests, reviews, Actions and releases. This workflow provides a traceable record of changes and allows automated checks to run before merge (GitHub, n.d.).

Table 8 Development and build tools

Tool	Project use
Git and GitHub	Commits, branches, Issues, Pull Requests, review and releases
GitHub Actions	Run npm test on pushes and Pull Requests
Node.js 22	HTTP server, crypto, built-in test runner and SQLite API
SQLite	Executable relational runtime database
Docker	Reproducible application container
Mermaid Live Editor	Editable online class, sequence and ER diagrams
GitHub Pages workflow	Static project documentation deployment support

Pull Request 27 corrected spending-trend precision. Pull Request 29 corrected the documented test total. Pull Request 30 recorded a formal teammate approval. Pull Request 31 removed an accidental nested repository reference. Pull Request 32 archived weekly coursework. Pull Request 34 implemented the report-interface correction. Pull Requests 35 and 36 aligned the acceptance and final-report documentation.

```
--base main \
--head docs/add-code-review-record \
--title "Document final code review workflow" \
--body "This pull request records the final GitHub workflow and corrects the application run instructions.

- Document PR #27 and PR #29
- Record 41 passed automated tests
- Explain GitHub Actions for push and pull request events
- Update the demo instructions for the Node.js and SQLite application"

Creating pull request for docs/add-code-review-record into main in lbw138286/cp3407

https://github.com/lbw138286/cp3407/pull/30
(base) jiangjinhua@MacBook-Air CP3407-Final-Repo % cd ~/Downloads/CP3407-Final-Repo
gh pr view 30 --json state,reviewDecision,reviews,url
{
  "reviewDecision": "APPROVED",
  "reviews": [
    {
      "id": "PRR_kwDOSHrNMBAAAAAISIROA",
      "author": {
        "login": "lbw138286"
      },
      "authorAssociation": "OWNER",
      "body": "I reviewed the final version-control documentation and application run\\ninstructions. The changes are accurate and consistent with the current\\nNode.js, SQLite and GitHub Actions implementation.",
      "submittedAt": "2026-08-04T05:13:09Z",
      "includesCreatedEdit": false,
      "reactionGroups": {},
      "state": "APPROVED",
      "commit": {
        "oid": "f4d29488dc75002847080dbbe57b0dd85d10da4b"
      }
    }
  ],
  "state": "OPEN",
  "url": "https://github.com/lbw138286/cp3407/pull/30"
}
(base) jiangjinhua@MacBook-Air CP3407-Final-Repo % gh pr merge 30 --merge --delete-branch
✓ Merged pull request lbw138286/cp3407#30 (Document final code review workflow)
remote: Enumerating objects: 1, done.
remote: Counting objects: 100% (1/1), done.
remote: Total 1 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (1/1), 932 bytes | 310.00 KiB/s, done.
From https://github.com/lbw138286/cp3407
 * branch          main      -> FETCH_HEAD
 e067c8e..9205918  main      -> origin/main
Updating e067c8e..9205918
Fast-forward
 docs/version-control-and-tools.md | 25 ++++++
 1 file changed, 23 insertions(+), 2 deletions(-)
✓ Deleted local branch docs/add-code-review-record and switched to branch main
✓ Deleted remote branch docs/add-code-review-record
(base) jiangjinhua@MacBook-Air CP3407-Final-Repo % git switch main
git pull origin main
git status
Already on 'main'
Your branch is up to date with 'origin/main'.
From https://github.com/lbw138286/cp3407
 * branch          main      -> FETCH_HEAD
Already up to date.
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
(base) jiangjinhua@MacBook-Air CP3407-Final-Repo %
```

Figure 7 Pull Request, merge and clean-branch evidence

9 Acceptance Feedback and Corrective Action

During the final demonstration on 4 August 2026, Dasheng LIU stated that the data presentation was not convenient for users and used an overly complex method. The monthly report and spending trend information were mainly shown as raw JSON. The calculations were correct, but the display was more suitable for developers than normal users.

9.1 Corrective Action

Number	Statement
1	Raw JSON was removed as the primary report output.
2	Total-spent, expense-count and top-category summary cards were added.
3	Currency values and expense records were placed in a formatted table.
4	Category and monthly comparison bars were added.
5	Direction, change, period and latest-month summaries were added.
6	Clear guidance was added when there was not enough data for trend analysis.
7	Technical JSON was retained only as an optional expandable section.

The corrective action was implemented through Pull Request 34, verified by GitHub Actions and manual browser testing, and published in release v1.0.1. The original outcome remains recorded as Changes required because a second formal lecturer decision was not received. The completed response is documented without claiming a new acceptance decision.

Reports and Trends

Clear summaries, tables and visual comparisons for normal users.

Monthly Report

Spending Trend

Spending trend analysis generated.

DIRECTION

More data needed

OVERALL CHANGE

\$0.00

PERIOD

Insufficient data

LATEST MONTH

No monthly totals

At least two monthly totals are required.

Monthly Spending Comparison

Add expenses in at least two different months to compare spending trends.

► Technical JSON details

Figure 8 Report and spending-trend interface after the usability correction

10 Persistence and Deployment

10.1 SQLite Persistence Verification

The Node.js server was stopped and restarted without deleting the SQLite database.

The same account could log in after restart, and the previously created user, expense and report data remained available. Runtime database files are excluded from Git through .gitignore.

Table 9 SQLite persistence verification

Stage	Observed evidence	Result
Before restart	Two expense records and cross-month trend data were visible	Passed
Server stop and start	Port was released and Node.js restarted from the correct repository	Passed

Stage	Observed evidence	Result
After restart	The same account logged in and stored SQLite data remained available	Passed

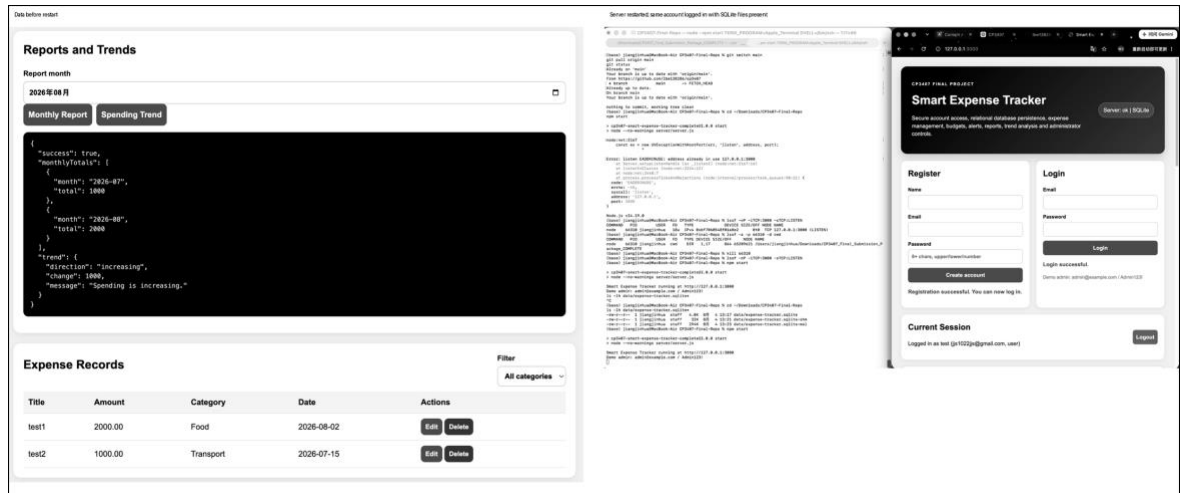


Figure 9 Data before restart and successful login after restarting the server

10.2 Deployment Status

Table 10 Deployment and release status

Item	Status	Evidence or limitation
Local application demo	Complete	npm start at http://127.0.0.1:3000
Docker packaging	Complete	Dockerfile and reproducible container configuration
Render deployment blueprint	Prepared	Node.js and SQLite persistent-disk configuration
GitHub Actions CI	Complete	Push and Pull Request checks pass

Item	Status	Evidence or limitation
Release management	Complete	v1.0.0 and v1.0.1 retained
Public Node.js URL	Not claimed	Requires a repository-owner hosting account
GitHub Pages URL	Workflow prepared	Published address must be verified after Pages is enabled

11 Conclusion

The final Smart Expense Tracker provides a modern browser interface, relational SQLite persistence, secure server-side authentication, user and administrator functions, 41 passing automated tests, 15 passing system tests and a documented Agile development history. The project also includes architecture, database and sequence designs, defect records, GitHub collaboration evidence and release management.

The project responded directly to the lecturer feedback. The original raw-JSON report presentation was replaced with summary cards, tables, formatted values, comparison bars and clear guidance for insufficient data. The correction is traceable through Pull Request 34 and release v1.0.1. This response strengthens the implementation, usability, testing and technical-writing evidence expected in the HD rubric.

12 References

GitHub. (n.d.). GitHub Docs. Retrieved August 5, 2026, from <https://docs.github.com/>

Jiang, J., Zhu, Z., & Fu, Y. (2026). Smart Expense Tracker (Version 1.0.1) [Computer software]. *GitHub*. <https://github.com/lbw138286/cp3407>

Mermaid. (n.d.). Mermaid documentation. Retrieved August 5, 2026, from <https://mermaid.js.org/>

Node.js. (n.d.). Node.js documentation. Retrieved August 5, 2026, from <https://nodejs.org/docs/latest/api/>

SQLite Consortium. (n.d.). SQLite documentation. Retrieved August 5, 2026, from <https://www.sqlite.org/docs.html>